

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 9 - Formative Assessment 3

Formative Assessment (FA-3) - Answer Key

Class: Class 9

Assessment Type: Formative Assessment (FA-3)

Total Marks: 30

Duration: 90 minutes

Topics Covered: SQL - Database Management

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

- Answer:** b) JOIN
 - Explanation:** The JOIN clause is used to combine rows from two or more tables based on a related column. UNION combines results of two SELECT statements, not rows from tables.
- Answer:** b) Counts only distinct departments
 - Explanation:** COUNT(DISTINCT department) counts only unique (non-duplicate) department values, ignoring NULLs.
- Answer:** c) ALTER
 - Explanation:** ALTER is a DDL (Data Definition Language) command used to modify table structure. SELECT, INSERT, and UPDATE are DML commands.
- Answer:** b) Pattern matching
 - Explanation:** The LIKE operator is used for pattern matching in SQL using wildcards % (any characters) and _ (single character).
- Answer:** d) Both B and C
 - Explanation:** COUNT(column_name) and SUM(column_name) both ignore NULL values. COUNT(*) counts all rows including NULLs.

Section II: Short Answer (5 x 2 = 10 marks)

- Write a SQL query to create a table Library.**

- Answer:**

```
CREATE TABLE Library (  
    book_id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(100) NOT NULL,  
    author VARCHAR(50) NOT NULL,  
    genre VARCHAR(30),  
    price DECIMAL(10, 2),  
    available BOOLEAN DEFAULT TRUE  
);
```

- Marking:** 1 mark for correct column definitions and data types, 1 mark for PRIMARY KEY, AUTO_INCREMENT, and DEFAULT constraints.
- Differentiate between WHERE and HAVING clauses.**
 - Answer:** WHERE filters rows before grouping (cannot use aggregate functions). HAVING filters groups after GROUP BY (can use aggregate functions).
 - Marking:** 1 mark for explaining WHERE with example, 1 mark for explaining HAVING with example.
 - Examples:**

```
SELECT department, AVG(salary) FROM Employees WHERE age > 25 GROUP BY department
HAVING AVG(salary) > 50000;
```

3. Write SQL queries for Students(roll, name, class, section, marks).

• Answer:

```
-- Students whose name starts with 'A'
```

```
SELECT * FROM Students WHERE name LIKE 'A%';
```

```
-- Students with marks between 50 and 80
```

```
SELECT * FROM Students WHERE marks BETWEEN 50 AND 80;
```

```
-- Students in sections 'A' or 'B'
```

```
SELECT * FROM Students WHERE section IN ('A', 'B');
```

```
-- Count students in each class
```

```
SELECT class, COUNT(*) AS student_count FROM Students GROUP BY class;
```

• Marking: 0.5 marks per correct query.

4. What is a Foreign Key?

- **Answer:** A Foreign Key is a column in one table that references the Primary Key of another table. It maintains referential integrity by ensuring that values in the foreign key column must exist in the referenced table or be NULL.

- **Marking:** 1 mark for definition, 1 mark for example showing referential integrity (e.g., Results.roll_no references Students.roll_no).

5. Write a SQL query to display the second highest salary.

• Answer:

```
SELECT MAX(salary) AS second_highest
```

```
FROM Employees
```

```
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

- **Marking:** 1 mark for correct subquery approach, 1 mark for explanation of logic (finding max of salaries less than the maximum).

Section III: Long Answer (5 x 1 = 5 marks)

1. Write SQL queries for the given schema.

• Answer:

```
-- Percentage of each student
```

```
SELECT s.roll_no, s.name,
```

```
ROUND(SUM(m.marks_obtained) * 100.0 / SUM(sub.max_marks), 2) AS percentage
```

```
FROM Students s
```

```
JOIN Marks m ON s.roll_no = m.roll_no
```

```
JOIN Subjects sub ON m.sub_id = sub.sub_id
```

```
GROUP BY s.roll_no, s.name;
```

```
-- Subject with highest average score
```

```
SELECT sub.sub_name, ROUND(AVG(m.marks_obtained), 2) AS avg_score
```

```
FROM Subjects sub
```

```
JOIN Marks m ON sub.sub_id = m.sub_id
```

```
GROUP BY sub.sub_id, sub.sub_name
```

```
ORDER BY avg_score DESC
```

```
LIMIT 1;
```

```
-- Students who scored above 90% in any subject
```

```

SELECT DISTINCT s.name, sub.sub_name, m.marks_obtained, sub.max_marks
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
JOIN Subjects sub ON m.sub_id = sub.sub_id
WHERE (m.marks_obtained * 100.0 / sub.max_marks) > 90;

-- Class topper
SELECT s.class, s.name, SUM(m.marks_obtained) AS total_marks
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
GROUP BY s.class, s.name
ORDER BY total_marks DESC
LIMIT 1;

-- Rank list by total marks
SELECT s.roll_no, s.name, s.class,
       SUM(m.marks_obtained) AS total_marks,
       RANK() OVER (ORDER BY SUM(m.marks_obtained) DESC) AS rank_position
FROM Students s
JOIN Marks m ON s.roll_no = m.roll_no
GROUP BY s.roll_no, s.name, s.class
ORDER BY rank_position;

```

- **Marking:** 1 mark per correct query (5 queries = 5 marks).

Part B: Hands-on Practical Lab Task (5 marks)

- **Answer:**

```

import sqlite3

# Connect to database
conn = sqlite3.connect('school.db')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE IF NOT EXISTS School (
    roll INTEGER PRIMARY KEY,
    name TEXT,
    class TEXT,
    section TEXT
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Results (
    roll INTEGER,
    sub1 INTEGER, sub2 INTEGER, sub3 INTEGER, sub4 INTEGER, sub5 INTEGER,
    FOREIGN KEY (roll) REFERENCES School(roll)
)''')

# Insert data
students = [
    (1, 'Amit', '9', 'A', 85, 78, 92, 88, 76),
    (2, 'Priya', '9', 'B', 92, 85, 88, 95, 90),
    (3, 'Rahul', '9', 'A', 68, 72, 65, 70, 58),
    (4, 'Sneha', '9', 'B', 88, 91, 85, 82, 87),
    (5, 'Vikram', '9', 'A', 55, 48, 62, 50, 45),
    (6, 'Ananya', '9', 'B', 95, 92, 98, 90, 94),

```

```

        (7, 'Deepak', '9', 'A', 72, 68, 75, 70, 65),
        (8, 'Meera', '9', 'B', 81, 84, 79, 86, 82),
    ]

    for s in students:
        cursor.execute('INSERT OR IGNORE INTO School VALUES (?, ?, ?, ?)', (s[0], s[1],
s[2], s[3]))
        cursor.execute('INSERT OR IGNORE INTO Results VALUES (?, ?, ?, ?, ?, ?)', (s[0],
s[4], s[5], s[6], s[7], s[8]))

    conn.commit()

    # Average marks per student
    print("=== Average Marks per Student ===")
    cursor.execute('''SELECT s.name, ROUND(AVG(r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0,
2) as avg
        FROM School s JOIN Results r ON s.roll = r.roll GROUP BY s.roll''')
    for row in cursor.fetchall():
        print(f"{row[0]}: {row[1]}")

    # Subject with highest class average
    print("\n=== Subject with Highest Class Average ===")
    cursor.execute('''SELECT 'Sub1' as subject, AVG(sub1) as avg FROM Results
        UNION ALL SELECT 'Sub2', AVG(sub2) FROM Results
        UNION ALL SELECT 'Sub3', AVG(sub3) FROM Results
        UNION ALL SELECT 'Sub4', AVG(sub4) FROM Results
        UNION ALL SELECT 'Sub5', AVG(sub5) FROM Results
        ORDER BY avg DESC LIMIT 1''')
    row = cursor.fetchone()
    print(f"{row[0]}: {row[1]:.2f}")

    # Students needing improvement
    print("\n=== Students Needing Improvement (Avg < 60%) ===")
    cursor.execute('''SELECT s.name, ROUND((r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0, 2)
as avg
        FROM School s JOIN Results r ON s.roll = r.roll
        WHERE (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 < 60''')
    for row in cursor.fetchall():
        print(f"{row[0]}: {row[1]}")

    # Grade assignment
    print("\n=== Grade Assignment ===")
    cursor.execute('''SELECT s.name, (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 as avg,
        CASE
            WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 90 THEN 'A'
            WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 75 THEN 'B'
            WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 60 THEN 'C'
            WHEN (r.sub1+r.sub2+r.sub3+r.sub4+r.sub5)/5.0 >= 45 THEN 'D'
            ELSE 'F'
        END as grade
        FROM School s JOIN Results r ON s.roll = r.roll''')
    for row in cursor.fetchall():
        print(f"{row[0]}: {row[1]:.2f} -> {row[2]}")

    conn.close()

```

- **Marking:** 1 mark for database connection and table creation, 1 mark for inserting data, 1 mark for average calculation, 1 mark for subject with highest average, 1 mark for grade assignment logic.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)